

Aplikacja SIMP (Sensor IoT Management Protocol)

Kamil Janik, Tomasz Kapusta

1. Opis funkcjonalny aplikacji

- **Co robi aplikacja?**

System jest rozwiązaniem do bezpiecznego, dwukierunkowego zarządzania siecią urządzeń pomiarowych. Odbiera na serwerze i przechowuje w pamięci operacyjnej periodyczną telemetrię oraz krytyczne powiadomienia alarmowe. Aplikacja z poziomu konsoli serwera pozwala również na zdalne symulowanie wysyłania komend do urządzeń.

- **Jaki problem użytkownika rozwiązuje?**

- Niezawodne zbieranie danych z urządzeń o niskim poborze mocy działających na słabych, zrywających się łączach radiowych.
- Ochrona komunikacji przed nieautoryzowanym dostępem przy użyciu uwierzytelniania wbudowanego w protokół.
- Zabezpieczenie danych przed utratą w przypadku chwilowej awarii.

- **Kto jest użytkownikiem (aktorzy)?**

- **Sensor (klient, strona aktywna):** Samodzielne urządzenie pomiarowe inicjujące każde połączenie TCP.
- **Serwer (strona pasywna):** Centralny punkt gromadzący dane, wyłącznie nasłuchujący połączeń i realizujący logikę autoryzacji.
- **Administrator (użytkownik pośredni):** Osoba obserwująca standardowe wyjście (konsolę/logi) serwera, w celu monitorowania działania sieci i odczytywania napływających alarmów.

2. Architektura rozwiązania

- **Model:** Aplikacja działa w scentralizowanym modelu **klient-serwer (wiele-do-jednego)**. Sensory łączą się z jednym serwerem.

- **Komponenty:**

1. **Oprogramowanie Sensora (Klient CLI):** Aplikacja symulująca sensor, zawierająca stos TCP/TLS (OpenSSL), logikę buforowania danych oraz zapisaną w kodzie konfigurację (**device_id**, **hasło**).
2. **Serwer Zbierający (Backend):** Usługa nasłuchująca domyślnie na porcie 8883. Zapewnia obsługę wielu klientów, kończenie tunelu TLS 1.3 oraz dekodowanie binarnych nagłówek (15 bajtów) i payloadów w formacie big-endian.
3. **Pamięć Operacyjna Serwera (In-memory Storage):** Struktury danych w pamięci RAM przechowujące predefiniowane pary dozwolonych **device_id** i **password** oraz tablice logujące odebraną w trakcie działania programu telemetrię. Dane znikają po wyłączeniu serwera.

- **Przepływ danych między komponentami:**

1. Sensor inicjuje połączenie TCP na port 8883 serwera.
2. Zestawiany jest szyfrowany kanał TLS.
3. Sensor przesyła ramki **HELLO** i **AUTH** , a serwer weryfikuje poświadczenia, sprawdzając zahardkodowaną tablicę autoryzowanych urządzeń w pamięci.
4. Po wydaniu **session_token** (stan **ACTIVE**), komunikaty **TELEMETRY** trafiają do serwera, który wypisuje je w konsoli i dopisuje do lokalnej tablicy w pamięci RAM.

3. Przypadki użycia (Use Cases)

UC1: Rejestracja i uwierzytelnianie sensora

- **Cel:** Sensor uzyskuje autoryzowany dostęp do serwera i otrzymuje **session_token**.
- **Aktor:** Sensor.
- **Warunki wstępne:** Sensor ma wgrane: **device_id**, pre-shared key, fingerprint cert. serwera.
- **Scenariusz główny:** Sensor otwiera połączenie TCP na port 8883. TLS handshake - sensor weryfikuje fingerprint certyfikatu serwera. Sensor wysyła **HELLO** (**device_id**: 8 B). Sensor wysyła **AUTH** (**password_len** + **password**). Serwer weryfikuje (**device_id**, **hash(password)**) w bazie. Serwer generuje losowy **session_token** i odsyła **AUTH_OK**. Sensor zapisuje token — używa go w każdej kolejnej ramce.
- **Scenariusz alternatywny (Błąd sesji):** Złe hasło → **AUTH_FAIL** → serwer zamyka TCP; po 3 próbach IP zablokowane 300s.
- **Wynik końcowy:** Sensor w stanie **ACTIVE**, gotowy do wysyłania danych.

UC2: Standardowe raportowanie telemetrii

- **Cel:** Przekazanie i zalogowanie w pamięci cyklicznych pomiarów czujnika.
- **Aktor:** Sensor, Serwer.
- **Warunki wstępne:** Stan sesji **ACTIVE** (po wydaniu **session_token** w **AUTH_OK**).
- **Scenariusz główny:** Sensor generuje i koduje ramkę **TELEMETRY** podając **sensor_type**, bieżący czas (Unix timestamp) oraz wartość float i wysyła ją na serwer. Serwer waliduje token sesji, dekoduje payload i dodaje pomiar do struktury w pamięci operacyjnej, wypisując log w konsoli.
- **Scenariusz alternatywny (Błąd sesji):** Sensor wysyła nieznany serwerowi **session_token**. Serwer odrzuca ramkę błędem **0x02 AUTH_INVALID** i zamyka połączenie.
- **Wynik końcowy:** Pomiar jest zapisany w pamięci procesu serwera i widoczny dla administratora.

UC3: Obsługa stanu alarmowego (Przekroczenie progu)

- **Cel:** Szybkie poinformowanie systemu centralnego o krytycznym zdarzeniu.
- **Aktor:** Sensor, Serwer.
- **Warunki wstępne:** Stan sesji **ACTIVE**.
- **Scenariusz główny:** Sensor rejestruje anomalię, tworzy ramkę **ALERT** z włączoną flagą bitową **ACK_REQUIRED** (bit 0 = 1). Ramka trafia na Serwer. Serwer loguje zdarzenie na czerwono w konsoli i odpowiada wiadomością **ACK** odsyłając odpowiedni **ack_seq**.

- **Scenariusz alternatywny (Brak potwierdzenia):** Sensor nie otrzymuje komunikatu **ACK** przez 10 sekund. Sensor ponawia wysyłkę maksymalnie 3 razy, następnie generuje **ERROR 0x04 TIMEOUT**.
- **Wynik końcowy:** Serwer odnotowuje alarm i pomyślnie potwierdza jego odbiór do sensora.

UC4: Zdalna zmiana konfiguracji czujnika

- **Cel:** Wysłanie komendy z serwera do sensora z żądaniem zmiany działania (np. zmiana interwału).
- **Aktor:** Serwer, Sensor.
- **Warunki wstępne:** Stan sesji **ACTIVE**.
- **Scenariusz główny:** Serwer tworzy ramkę **COMMAND** zawierającą **cmd_id**, unikalne **cmd_seq** i wymusza flagę **ACK_REQUIRED**. Sensor odbiera komendę, aktualizuje swoje zmienne wewnętrzne i odsyła do serwera wiadomość **ACK** z przypisanym numerem sekwencyjnym.
- **Scenariusz alternatywny (Duplikat):** Serwer wysyła komendę ponownie z powodu opóźnień. Sensor rozpoznaje duplikat dzięki identyfikatorowi **cmd_seq** w ramach tej samej sesji i odsyła sam **ACK** ignorując ponowną zmianę parametrów.
- **Wynik końcowy:** Sensor dostosowuje swoje parametry działania.

UC5: Utrata zasięgu i przywracanie łączności z buforowaniem

- **Cel:** Uniknięcie utraty danych podczas czasowej niedostępności sieci.
- **Aktor:** Sensor, Serwer.
- **Warunki wstępne:** Stan sesji **ACTIVE**, następuje awaria łącza.
- **Scenariusz główny:** Sensor wysyła **PING**. Brak odpowiedzi **PONG** w ciągu 5 sekund skutkuje próbą ponowienia, a następnie zamknięciem gniazda TCP. Sensor buforuje do 100 rekordów w pamięci lokalnej. Mechanizm exponential backoff wymusza czekanie 5s, 10s, 20s, 30s przed nowymi próbami. Po pomyślnym nowym cyklu uwierzytelniania, sensor opróżnia bufor, wysyłając zaległe dane **TELEMETRY** z historycznymi timestampami.
- **Wynik końcowy:** Pełna historia pomiarów ostatecznie dociera do serwera.

UC6: Ochrona przed atakiem Brute-force

- **Cel:** Obrona serwera przed atakiem słownikowym/zgadywaniem pre-shared key.
- **Aktor:** Atakujący, Serwer.
- **Warunki wstępne:** Nawiązane połączenie TCP i TLS (stan **AUTHENTICATING**).
- **Scenariusz główny:** Atakujący podaje hasło niezgodne z danymi zahardkodowanymi w pamięci serwera. Serwer zwraca **AUTH_FAIL 0x02** i zamyka połączenie. Sytuacja powtarza się 3 razy z tego samego IP w ciągu 60 sekund. Kolejna próba nawiązania TCP skutkuje wysłaniem przez serwer **ERROR 0x08 RATE_LIMITED** i zablokowaniem IP na 300 sekund.
- **Wynik końcowy:** Zasoby serwera są chronione przed atakiem typu Denial of Service.

UC7: Podgląd historii danych przez administratora

- **Cel:** Administrator przegląda historyczne odczyty i alerty.
- **Aktor:** Serwer.
- **Warunki wstępne:** Serwer działa; w bazie istnieją rekordy.
- **Scenariusz główny:** Administrator wpisuje: show DEV001 TEMP last 60. cli_module wywołuje storage_module z filtrami: device_id, typ, zakres czasu. storage_module wykonuje zapytanie do SQLite i zwraca listę rekordów. cli_module wyświetla tabelę: timestamp | sensor_type | value. Administrator może też wpisać: show alerts - lista alertów
- **Scenariusz alternatywny (brak danych):** Brak danych dla podanego urządzenia - 'No data found for DEV001'. Nieprawidłowa składnia komendy CLI - wypisanie help.
- **Wynik końcowy:** Administrator widzi historię odczytów i może podjąć decyzję o komendzie.

4. Mapowanie aplikacji na protokół

Funkcja aplikacji	Komunikat SIMP	Kierunek	UC
Nawiązanie sesji TLS i identyfikacja	HELLO (0x01)	S → Srv	UC1
Podanie hasła	AUTH (0x02)	S → Srv	UC1
Wydanie tokenu sesji	AUTH_OK (0x03)	Srv → S	UC1
Odmowa dostępu	AUTH_FAIL (0x04)	Srv → S	UC1
Periodyczny odczyt czujnika	TELEMETRY (0x05)	S → Srv	UC2
Przekroczenie progu alarmowego	ALERT (0x06)	S → Srv	UC3
Potwierdzenie alertu	ACK (0x08)	Srv → S	UC3
Komenda administratora	COMMAND (0x07)	Srv → S	UC4
Potwierdzenie wykonania komendy	ACK (0x08)	S → Srv	UC4
Weryfikacja żywotności połączenia	PING/PONG (0x09/0x0A)	Obukier.	UC5
Informacja o błędzie protokołu	ERROR (0x0B)	Obukier.	wszystkie
Czyste zamknięcie sesji	BYE (0x0C)	Obukier.	UC5

5. Wymagania niefunkcjonalne

- **Bezpieczeństwo**

Wymaganie	Realizacja
Poufność komunikacji	TLS 1.3 — szyfrowanie całego kanału (AES-256-GCM)
Uwierzytelnienie serwera	Sensor weryfikuje fingerprint SHA-256 certyfikatu serwera
Uwierzytelnienie sensora	HELLO+AUTH: device_id + PBKDF2(password)
Autoryzacja operacji	session_token sprawdzany przy każdej ramce (O(1))
Ochrona przed brute-force	Rate-limit: 3 próby AUTH / 60 s / IP; ban 300 s
Brak wycieku informacji w błędach	ERROR zawiera ogólny kod, nie szczegóły wewnętrzne

- **Wydajność**

Parametr	Wymaganie	Uzasadnienie
Maks. równoczesnych sensorów	50	Jeden wątek per sensor; 50 wątków to rozsądny limit
Opóźnienie zapisu telemetry	< 50 ms	Szybki zapis, możliwy odczyt równoległy
Opóźnienie odpowiedzi ACK	< 100 ms w LAN	Serwer odpowiada ACK natychmiast po walidacji ramki
Przepustowość	≥ 500 ramek/s	Jedna ramka TELEMETRY = ~18 B; 500 × 18 B = 9 KB/s

- **Niezawodność**

Scenariusz awarii	Zachowanie systemu
Sensor traci łączność	Reconnect z exponential backoff; dane buforowane lokalnie (ring 100 rek.)
Serwer restartuje	Po restarcie wczytuje bazę; stare sesje wygasłe — sensory ponownie autoryzują

Zapis do pamięci nie powiódł się	Serwer loguje błąd, wysyła ERROR 0x06 do sensora; sesja może trwać
Sensor wysyła zniekształconą ramkę	Serwer wysyła ERROR 0x01 i zamyka sesję; sensor reconnectuje

- **Skalowalność**

Parametr	Wymaganie
Maksymalny czas trwania sesji w stanie CONNECTED bez AUTH	10s
Maksymalna ilość otwartych sesji z jednego adresu IP	5

6. Plan implementacji i testowania

- **Zakres MVP (Minimum Viable Product)**

MVP to wersja systemu spełniająca minimalne wymagania Etapu 3. Priorytet: poprawna komunikacja SIMP, bezpieczeństwo, obsługa błędów.

Lp.	Funkcja MVP	Komponent	Priorytet
M1	Serwer nasłuchuje TCP+TLS, przyjmuje jedno połączenie	simp_server + TLS	Krytyczny
M2	Obsługa HELLO+AUTH - wydanie session_token	auth_module	Krytyczny
M3	Odbieranie TELEMETRY i zapis do pliku CSV	storage_module (CSV)	Krytyczny
M4	Sensor wysyła HELLO+AUTH+TELEMETRY w pętli	sensor_client	Krytyczny
M5	Obsługa wielu równoczesnych sensorów (wątki)	session_manager	Wysoki
M6	Obsługa ALERT z ACK i retry po stronie klienta	sensor_client	Wysoki
M7	Wysyłanie COMMAND z CLI i odbiór ACK	cli_module	Wysoki
M8	Reconnect z exponential backoff	sensor_client	Wysoki
M9	Obsługa błędów: FORMAT_ERROR,	simp_server	Wysoki

	AUTH_FAIL, TIMEOUT		
M10	Podgląd historii przez CLI (UC7)	cli_module	Średni
M11	Rate-limiting prób AUTH	auth_module	Średni

- Plan testów

A. Testy funkcjonalne

ID testu	Opis	Oczekiwany wynik
TF1	Poprawna sesja: HELLO→AUTH→TELEMETRY×5→ BYE	Serwer zapisuje 5 rekordów; graceful shutdown
TF2	Złe hasło w AUTH	AUTH_FAIL; TCP zamknięty; log WARN
TF3	Wysłanie ALERT — serwer wysłał ACK	Klient nie ponawia; zdarzenie w bazie
TF4	CLI: cmd DEV001 SET_INTERVAL 30	Sensor zmienia interwał; ACK odebrany
TF5	CLI: show DEV001 TEMP last 60	Wypisana tabela rekordów z ostatnich 60s
TF6	Niezainicjalizowane TELEMETRY (brak AUTH)	Serwer odpowiada ERROR 0x05

B. Testy błędów i odporności

ID testu	Opis	Oczekiwany wynik
TE1	Zabicie procesu klienta w trakcie sesji	Serwer wykrywa timeout 60 s, czyści sesję
TE2	Symulacja zerwania sieci (iptables DROP)	Klient reconnectuje z backoff; dane z bufora dostarczone
TE3	Ramka z payload_len=9999	Serwer: ERROR 0x07, zamknięcie sesji; bez crashu
TE4	Nieznany type=0xFF w ramce	Serwer: ERROR 0x01, zamknięcie sesji
TE5	3 próby AUTH z złym hasłem z jednego IP	4. próba: ERROR 0x08; IP zablokowane
TE6	Duplikat COMMAND (ten sam cmd_seq)	Sensor ignoruje duplikat, wysłał ACK bez wykonania

C. Testy bezpieczeństwa

ID testu	Opis	Oczekiwany wynik
TS1	Połączenie bez TLS (zwykły TCP)	Serwer odmawia — nie ma handshake TLS
TS2	Fałszywy certyfikat serwera (self-signed, inny fingerprint)	Klient odrzuca połączenie przed AUTH
TS3	Ramka z cudzym (ale ważnym) session_token	Serwer: ERROR 0x02, zamknięcie sesji
TS4	Replay: odtworzenie nagranej sesji	TLS anti-replay odrzuca; nowy handshake wymagany

- Podział pracy w zespole

Osoba	Odpowiedzialność	Artefakty (pliki)
Kamil Janik	Generowanie certyfikatów, nasłuch TLS, sesje, auth, storage, konsola CLI, testy serwera	simp_server.py, tls_server.py, session.py, auth.py, storage.py, cli.py, test_auth.sh
Tomasz Kapusta	Weryfikacja TLS, budowanie ramek, symulator czujnika, reconnect, buforowanie, testy klienta	simp_client.py, tls_client.py, protocol.py, sensor_sim.py, reconnect.py, test_telemetry.sh
Wspólnie	Ustalenie formatu ramek, testy integracyjne, błędów i bezpieczeństwa, przygotowanie demo	simp_protocol.py, test_errors.sh, test_security.sh, README.md